Azure DevOps → GitHub Actions Migration Blueprint (Enterprise)

This migration blueprint is a complete, executable plan to move from Azure DevOps (Repos, Pipelines, Boards, Artifacts, Variable Groups, Service Connections) to GitHub Enterprise Cloud + GitHub Actions. It follows a phased, low-risk approach that separates strategy, design, execution, and validation; aligns with enterprise security & compliance; supports many teams and repos; and enables maintainability and scale.

---

1. Migration objectives and success criteria

Objectives

- Consolidate source control, CI/CD, packages and issues on GitHub Enterprise Cloud and GitHub Actions while preserving traceability, security, and delivery velocity.
- Replace Azure Pipeline CI/CD with GitHub Actions workflows; migrate repositories, packages and work items in a controlled manner.
- Implement modern, least-privilege credential model using OIDC, org-level governance, and IaC to ensure repeatability.
- Enable developer adoption through templates, reusable workflows, and training.

Success criteria (must be met before decommissioning ADO)

- Repo parity: 100% of planned repositories migrated with branches/tags/commit counts matching (± small allowed).
- CI parity: For a defined validation window, build/test pass-rate in GitHub Actions equals or exceeds ADO for representative builds (pilot + canary groups).
- Artifacts parity: Artifact checksums for binary outputs and published packages validated for migrated artifacts.
- Secrets & identity: SSO (SAML) + SCIM provisioning active; OIDC trust configured for cloud provider federations used by migrated pipelines.
- Security posture: Branch protection, required reviews, Dependabot/CodeQL (as appropriate) enabled; audit logs streaming to SIEM.
- Operational readiness: Runbooks, rollback procedures, and support channels in place; teams trained; monitoring/alerts configured.
- Cutover acceptance: Stakeholder sign-off after production pilot and run-in period.

Key acceptance KPIs

- Build success parity >= 95% across validated jobs
- Median pipeline runtime variance within ±20% after tuning
- No high-severity security findings introduced by migration
- Secret rotation & OIDC applied to >90% of production workflows

---

2. Current-state assessment framework

Purpose: Produce an inventory with metadata that drives scoping, prioritization, risk scoring

and automation.

Inventory columns (CSV/DB export)

- Repo name, size (GB), LFS usage, protected branches, commit count, primary language
- Pipelines count (YAML/classic), pipeline types (CI/CD/scheduled/manual), runtime images, self-hosted agents used, artifact outputs
- Service connections referenced (Azure subscriptions, registries, third-party)
- Variable groups referenced, secrets referenced, owners (primary + backup), business criticality (P0/P1/P2)
- Boards: work item count, custom fields used, attachments, integrations
- Packages: feed names, types (NuGet/npm/Maven/Python/Docker), versions, downstream consumers
- Compliance tags: regulatory scope, retention requirements, data classification
- Migration risk score = weighted sum (complexity, criticality, LFS/size, proprietary tasks, custom boards)

Assessment steps

1. Run automated discovery: export ADO metadata via APIs (repos, pipelines, builds, artifacts, boards, service connections).
2. Enrich inventory with owner interviews and verification of runtime requirements (e.g., special hardware).
3. Score and bucket repos/pipelines into groups: Pilot (low-risk, representative), Non-prod (medium), Prod (high).
4. Identify dependencies across repos (submodules, packages) and cross-repo secrets usage.

Outputs

- Master inventory CSV and prioritized migration backlog
- Migration RACI per repo/team
- Pre-migration checklist per repo/pipeline

---

3. Target-state DevOps architecture (reference)

High-level components

- GitHub Enterprise Cloud organization(s) with defined teams and access model (SCIM-provisioned via Azure AD SSO).
- Repositories in GitHub, with standardized repo templates and CODEOWNERS.
- CI/CD: GitHub Actions workflows (reusable workflows & composite actions) stored at org-level and repo-level (.github/workflows).
- Runners: mix of GitHub-hosted runners for standard workloads and self-hosted runners for special hardware or network-limited builds.
- Secrets: OIDC federated credentials for cloud operations, GitHub Organization/Repository/Environment secrets for other secrets; optional integration with Azure Key Vault or HashiCorp Vault for high-sensitivity secrets.
- Packages: GitHub Packages / GHCR for container images and packages, with republishing pipelines.
- Issues & Projects: GitHub Issues and Projects (or Projects beta) replacing Boards; integration patterns for audit-case workflows.
- Governance: Terraform-managed org settings (provider: github), branch protection

templates, actions allowlist / enterprise policies.
- Security: Dependabot, CodeQL scanning, secret scanning, and audit logs shipped to SIEM.

Reference architecture diagram (logical)

- Azure AD (IdP) -> SAML SSO + SCIM -> GitHub Enterprise Org
- GitHub Actions -> OIDC -> Azure/AWS/GCP roles
- GitHub Audit Logs -> SIEM
- Reusable workflows in .github / internal action registry -> used by repo workflows
- Terraform IaC for org configuration + migration scripts (gh CLI/GraphQL) for per-repo tasks

---

4. Phased migration strategy and milestones

Phasing follows: Discovery & Inventory → Design & Standardization → Pilot Migration → Scaled Execution → Validation & Decommissioning

Phase 0 — Planning & Inventory (2–6 weeks)

- Goals: Complete inventory, governance design, SSO/SCIM configuration, migration governance board.
- Key tasks:
  - Create migration team (SRE/Platform, Security, App Owners, Infra, PM).
  - Run discovery and produce prioritized backlog.
  - Configure GitHub SSO (SAML) and SCIM with Azure AD; establish org structure (teams).
  - Define policies: branch protection baseline, Actions allowlist, environments and secrets policy.
  - Prepare IaC repository (Terraform) for org config.
- Milestones:
  - Inventory CSV + Migration backlog
  - Org pre-configured with SSO/SCIM and basic settings
  - Migration runbook drafted

Phase 1 — Design & Standardization (2–4 weeks, overlap with Phase 0)

- Goals: Define workflow standards, reusable workflows, security model, migration tooling.
- Key tasks:
  - Build org-level reusable workflows (CI, Test, Publish) and composite actions for common tasks.
  - Define naming conventions, branch-protection policies, CODEOWNERS and PR templates.
  - Design secrets & OIDC model: which workloads will use OIDC, which need Key Vault.
  - Create action allowlist and vet marketplace actions.
  - Prepare conversion templates and mapping cheat-sheets for tasks -> actions.

- Milestones:
  - Template repo with reusable workflows and sample converted pipelines
  - Security model and OIDC trust templates ready

Phase 2 — Pilot Migration (2–4 weeks)

- Goals: Migrate representative sample repos/pipelines; validate dual-run parity and security model.
- Selection: 5–10 non-critical repos representing languages, build sizes, and pipeline types.
- Key tasks:
  - Mirror repos (git --mirror + LFS handling) into GitHub.
  - Create PRs in repos with converted GitHub Actions workflows (CI first).
  - Configure OIDC or secrets and environment protections.
  - Run dual-run period: enable workflows in ADO and GitHub; collect metrics.
  - Migrate packages for pilot projects and update dependency configs.
  - Perform validation: build/test parity, artifact checksum checks, secrets access verification.
- Milestones:
  - Pilot validation report (KPIs met or remedial actions identified)
  - Updated conversion scripts/templates

Phase 3 — Scaled Execution (non-prod then prod, 4–12+ weeks depending on scale)

- Goals: Batch migration of remaining non-production then production workloads with automation.
- Strategy: Batch by teams or business domain; start with non-prod, then sequential production cutovers with approvals.
- Key tasks:
  - Automate repo creation and mirror; generate PRs with converted workflows for owner review.
  - Migrate packages in staged manner (non-prod, then prod).
  - Gradually enable OIDC across workflows; rotate/retire service principals/secrets.
  - Use canary approach for production workflows: single repo cutover → 24–72 hour observation → next.
  - Runbook-driven approvals for production cutover.
- Milestones:
  - All non-prod repos migrated and stable
  - Production cutover complete for core services
  - Migration health dashboard in place

Phase 4 — Validation & Decommissioning (2–8 weeks)

- Goals: Validate entire estate, terminate ADO services, finalize docs and training.
- Key tasks:
  - Re-run validation suite: repo parity, build parity, artifact checksums, package resolution, security checks.

    - Collect audit logs and confirm SIEM ingestion for GitHub events.
    - Decommission ADO projects incrementally (locks before deletion) after backup.
    - Finalize training and handoff to Platform/DevOps operations teams.

- Milestones:
    - ADO decommission plan executed (per project)
    - Post-migration optimization roadmap created

---

5. CI/CD workflow design standards and templates

Standards (applies to all converted workflows)

- Authoring patterns:
    - Prefer reusable workflows (workflow_call) and composite actions for shared logic.
    - Keep workflow files < 500 LOC where possible; break complex flows into multiple workflows.
- Security:
    - Explicit permissions: set permissions at workflow level (permissions: contents: read, id-token: write only when needed).
    - Use OIDC to obtain cloud credentials; avoid long-lived secrets.
    - Limit concurrency where needed: concurrency: group and cancel-in-progress: true.
- Environments:
    - Use GitHub Environments for staging/prod with required reviewers and environment secrets.
- Secrets:
    - Use repo/env/org secrets; minimize org-level secrets and restrict access to specific repos.
- Artifact handling:
    - Use actions/upload-artifact and download-artifact for ephemeral artifact handoff.
    - Publish release artifacts to GitHub Releases or GitHub Packages/GHCR.
- Versioning:
    - Pin actionable external actions to full SHA or semver with known-good ranges.
- Observability:
    - Emit structured logs and use annotations for failures; integrate with central monitoring.

Minimal example: CI workflow (dotnet)

```
name: CI – build & test
```

```yaml
on:
  push:
    branches: [ main, 'release/*' ]
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  packages: write
  id-token: write   # for OIDC if publishing

jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        dotnet: [6.0, 7.0]

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v3
        with:
          dotnet-version: ${{ matrix.dotnet }}
      - run: dotnet restore
      - run: dotnet build --no-restore --configuration Release
      - run: dotnet test --no-build --verbosity normal
      - uses: actions/upload-artifact@v4
        with:
          name: build-${{ matrix.dotnet }}
          path: bin/Release
```

Reusable workflow snippet (called by repo-specific workflows)

```yaml
# .github/workflows/reusable-ci.yml
name: Reusable CI
on:
  workflow_call:
    inputs:
      dotnet-version:
        required: true
        type: string

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v3
        with:
          dotnet-version: ${{ inputs.dotnet-version }}
      - run: dotnet restore
      - run: dotnet build --configuration Release
```

Translation checklist (per pipeline)

- Map triggers (branch/tag) -> on: push/pull_request
- Map pool -> runs-on
- Replace tasks -> marketplace actions or script steps (pin SHAs)
- Replace variable groups -> secrets/env/organization variables
- Replace publish artifact -> actions/upload-artifact
- Add permissions and id-token where OIDC required
- Configure environment protection for deploy steps
- Add workflow-level concurrency and caching steps (actions/cache) as appropriate

Templates deliverables

- Org-level template repository with:
  - Reusable CI and CD workflows
  - Composite actions for common tasks (build/test/publish/deploy)
  - PR templates for converted workflow reviews
  - Linting config and example workflow_call usage

---

6. Security, identity, and secrets management model

Identity & access

- Provisioning: SAML SSO with Azure AD + SCIM provisioning enabled before mass migration.
- Teams & RBAC:
  - Map ADO groups to GitHub Teams with principle of least privilege.
  - Use team-based repo access; avoid wide org-level admin grants.
- Service accounts:
  - Avoid long-lived machine user PATs. Use OIDC federated credentials and short-lived tokens for cloud provider access.
  - If machine users are required, constrain via fine-grained PATs and rotate regularly.

Secrets model

- Prefer OIDC-based federated credential approach:
  - Configure GitHub Actions OIDC provider and federate to Azure AD/Azure for role-assumption; eliminates stored cloud secrets for most deployments.
- For remaining secrets:
  - Use Repository secrets for repo-specific values.
  - Use Environment secrets for deployment-specific secrets requiring approvals.
  - Use Organization secrets when necessary, with repository allow-lists.
- High-sensitivity secrets:
  - Integrate with external vault (Azure Key Vault/HashiCorp Vault) to fetch secrets

at runtime using short-lived auth.
    - Use self-hosted runners in private networks that can access Key Vault endpoints securely when required.

Permissions & least-privilege

- Use workflow permissions minimal model in YAML: only request contents: write/read, id-token: write only when needed.
- Use branch protection & CODEOWNERS to enforce PR reviews, required status checks, and signed commits policy if required.

Audit & logging

- Enable Enterprise audit logs and forward to SIEM (Azure Sentinel, Splunk). Ensure logs include secret changes, token issuance, and environment approval events.
- Configure logging for OIDC token issuance in Azure AD (sign-ins / federated credentials).

Secret migration and handling

- Never store plaintext secrets in migration scripts or PRs. Use secure transfer mechanisms (Key Vault-to-GH script that creates secrets via API using short-lived admin token).
- Rotate secrets immediately after migration or creation.
- For bulk secret population, use automation with audit trail (Terraform or GitHub REST API) and ensure admin approval.

Action vetting

- Create an allowlist/denylist for marketplace actions. Vet high-use actions and either pin to SHA or fork and vet internally. Consider internal action registry for critical tasks.

Compliance

- Document retention and artifact immutability settings; configure retention windows to meet regulatory needs.
- For regulated data in issues/attachments, ensure appropriate repo visibility and access controls.

---

7. Risk management and rollback strategy

Primary mitigation strategies

- Dual-run (parallel execution) until validation: keep ADO pipelines active and run GitHub Actions in parallel. Use flags/labels to denote authoritative runs.
- Canary cutover: migrate small set to production first and observe for a defined period.
- Maintain immediate rollback paths in runbook: re-enable ADO triggers, revert workflow files, and restore secrets.

Rollback playbook (example steps)

1. Detect issue (failed deployments, security exposure, build divergence).
2. Triage and assign severity.
3. If CI/regression: disable GitHub workflow trigger and re-enable ADO pipeline trigger;

notify teams.
4. If deployment incident:
    - If safe: run rollback via application-level feature flag or deployment tool (blue/green or revert).
    - If not: re-invoke ADO CD pipeline or use cloud control plane to revert deployment.
5. Rotate any secrets suspected to be compromised.
6. Capture incident details and do post-incident review.

Pre-cutover safety net

- Keep ADO project as read/write for a retention window (defined per team, e.g., 30–90 days) until final validation.
- Snapshot state: export ADO build logs, artifacts, and configuration before final decommission.

Risk register (sample)

- Missing task parity -> build failures -> Mitigation: manual remediation, custom composite actions, dual-run
- LFS migration failures -> broken history -> Mitigation: LFS verification scripts and GitHub support engagement
- Secrets leakage -> exposure -> Mitigation: secure transfer with KMS, immediate rotation and audit
- Permission misconfigurations -> data exposure -> Mitigation: pre-cutover permission audit and staged SCIM provisioning

---

8. Developer enablement and change management plan

Objectives

- Minimize developer friction and provide support during and after migration.

Components

- Communication plan:
    - Announce migration timeline, impacts, and support channels.
    - Weekly migration status updates and dashboards.
- Training:
    - Hands-on workshops: GitHub Actions authoring, reusable workflows, OIDC, secrets best practices.
    - Recorded sessions and written quick-start guides.
- Enablement artifacts:
    - Conversion playbook, cheat-sheets (mapping tasks -> actions).
    - Repository templates, example workflows for major stacks (dotnet, Node.js, Java, Python).
    - Migration PR template and checklist to help repo owners review converted

workflows.

- Support model:
  - Migration office hours and Slack/Teams channel for live support during cutover.
  - Dedicate platform SRE engineers to act as migration buddies for critical teams.
- Incentives:
  - Recognize early adopters, provide "migration credits" or allocate small dev time reimbursement for teams performing conversion.
- Governance for change:
  - Create a migration governance board for exceptions and risk approvals.
  - Establish a fast-track process for emergency rollback and pre-approved production cutover windows.

Deliverables for developers

- Repo-specific PR with converted workflow plus clear review checklist:
  - "Does this workflow require OIDC?" "Does it reference a secret?" "Is the action pinned?"
- Automated linters (GHAS pre-commit or CI checks) to enforce standards.
- Template library and internal action marketplace.

---

9. Metrics and KPIs to measure migration success

Operational KPIs (tracked per batch/repo)

- Build success rate comparison (ADO vs GitHub) — target: $>=95\%$ parity for validated builds
- Mean time to repair (MTTR) for build failures — target: $<=$ previous baseline
- Pipeline runtime variance — target: median within $\pm 20\%$
- Time to deploy (lead time from commit to deploy) — target: no regression from baseline

Security & compliance KPIs

- % workflows using OIDC for cloud access — target: $>=90\%$ for production deployments
- Number of secrets stored at org level vs repo/env — target: minimize org-level secrets
- Audit log completeness & SIEM ingestion latency — target: <5 minutes for critical events
- Dependabot/CodeQL coverage across repos — target: 100% enabled for relevant repos

Adoption metrics

- % repos with migrated default branch workflows enabled
- % teams trained
- Number of issues/tickets opened during migration per repo

Business KPIs

- Number of production incidents attributable to migration — target: 0 major incidents
- Time to decommission ADO components (post-validation) — tracked to closure

Dashboarding

- Create a migration dashboard showing inventory status, per-batch health, KPIs, and outstanding blockers. Use GitHub Issues/Projects for tracking and a BI tool connected to inventory CSVs and pipeline telemetry.

---

10. Post-migration optimization and governance guidance

Immediate post-migration tasks

- Consolidate reusable workflows and composite actions collected from teams into org library.
- Pin and vet marketplace actions; reduce external dependencies or fork critical actions.
- Tune runner types and scaling: move standard workloads to hosted runners; maintain self-hosted for special needs.
- Migrate CI caches and tune caching (actions/cache) for faster runs.

Governance (ongoing)

- IaC-driven org configuration: use Terraform for branch protection, team membership, environments and default workflows.
- Policy enforcement: enforce required checks, code owners, PR review policies and protected branches.
- Security automation: enable Dependabot, CodeQL, secret scanning and required status checks for security before merging.
- Periodic audit and refresh: quarterly review of secrets, OIDC trust relationships, and action allowlists.

Optimization roadmap

- Consolidate duplicative workflows, reduce tech debt in actions.
- Establish internal action registry and version policy.
- Implement automated migration for new repos using templates.
- Continuous improvement: collect metrics monthly and iterate.

Decommissioning ADO

- Use staged decommission:
    - Freeze writes to ADO after final migration approval for a defined period.
    - Archive and snapshot ADO data (repos, artifacts, boards).
    - Close ADO projects with documented reference and rollback pointers.
- Ensure all legal/regulatory retention needs are met prior to deletion.

---

11. Tooling, automation & deliverables

Essential tools

- git (mirror), git-lfs
- GitHub CLI (gh) and REST/GraphQL migration APIs
- GitHub Enterprise Importer (when needed)
- Terraform with GitHub provider for org config
- Azure DevOps Migration Tools for work items (only after testing)
- Custom scripts (Bash/PowerShell/Python) for orchestration, LFS migration, artifact checksum checks, and PR creation

Automations to build

- Repo migration orchestrator:
  - Create target repo, apply repo settings, branch protection, CODEOWNERS
  - Mirror git repo with LFS handling and verify parity
  - Create PR with converted workflow YAML
- Package republish pipelines for each package type
- Validation suite:
  - Repo parity checks (commit counts/branches)
  - Build comparison: run GitHub Actions and compare logs, artifact checksums to ADO outputs
  - Permissions audit report
- Secrets migration tool:
  - Securely ingest secrets from vault or ADO, create GH secrets via API with audit trail

Deliverables

- Migration inventory CSV
- Conversion repo: conversion scripts + sample converted workflows + PR templates
- Runbook (pre-cutover checklist, cutover steps, rollback steps, contacts)
- IaC repo for org configuration and policies
- Validation automation and migration dashboard
- Training materials and cheat sheets

---

12. Validation, testing and acceptance criteria (detailed)

Validation suite tests (automated)

- Repo validation:
  - Branches/Tags parity
  - Commit counts and last commit timestamps
  - LFS object counts and sizes
- Build validation:
  - Run converted GitHub workflow; compare compiled artifacts and checksums with ADO outputs

    - Test pass/fail parity
    - Performance baseline (runtime) comparison

- Package validation:

    - Publish/restore tests for packages republished to GitHub Packages/GHCR
    - Dependency resolution smoke tests in dependent repos

- Secrets & OIDC:

    - Test OIDC role assumption flows end-to-end
    - Verify secret-scoped access for environment-only secrets

- Permissions & governance:

    - Confirm SCIM-provisioned teams and repo access
    - Ensure branch protections and required checks are in place

- Operational:

    - Verify audit logs and SIEM ingestion for sample events
    - Confirm monitoring & alerts are triggered for expected events

Acceptance gates (example)

- Pilot acceptance: all pilot repos pass validation suite; fewer than X (e.g., 5%) repo owners report functional gaps.
- Non-prod acceptance (per-batch): validation tests pass; security checklist cleared.
- Prod acceptance: Production workflows have been run successfully over 3 production cycles (or 30 days) with no critical regression.

---

13. Work items / Boards migration guidance

Options

- Incremental: Keep Azure Boards integrated with GitHub Repos for a period while migrating repos; then migrate work items gradually using mapping tools.
- Direct import: Use Azure DevOps Migration Tools to map work items to GitHub Issues (expect schema changes and manual mapping for custom fields).
- Projects & Issue management: Adopt GitHub Projects (or the new Projects beta) to model workflows and custom fields where necessary.

Best practices

- Map fields to labels and milestones; preserve comments and attachments with explicit links where direct mapping isn't possible.
- For heavy/custom workflows, redesign in terms of GitHub Issue + Projects instead of attempting one-to-one mapping.
- Archive ADO Boards only after stakeholders confirm issue migration parity.

---

14. Risk mapping and common failure modes (summary)

Top risks and mitigations

- Build failures due to missing tasks or environment differences:
  - Mitigation: Dual-run, self-hosted runners for parity, and thorough toolchain validation.
- LFS and repo history loss:
  - Mitigation: LFS-aware mirroring with checksum verification and external backup prior to deletion.
- Secret mismanagement and exposure:
  - Mitigation: Secure secret transfer, immediate rotation, and use of OIDC for cloud credentials.
- Permission misconfiguration and data exposure:
  - Mitigation: SCIM-based provisioning, pre-cutover audit, and least-privilege team mapping.
- Package dependency breakage:
  - Mitigation: Staged republishing + dependency smoke-tests.

---

15. Sample timelines and milestones (example for medium enterprise: ~200 repos)

Estimated total: 3–6 months (depending on team size and complexity)

- Weeks 1–3: Phase 0 Planning & Inventory
- Weeks 3–6: Phase 1 Design & Standardization (templates & governance)
- Weeks 6–10: Phase 2 Pilot (5–10 repos)
- Weeks 11–18: Phase 3 Non-prod scaled execution (batches of 20–40 repos)
- Weeks 19–26: Phase 3 Production migration (sequentially per business domain)
- Weeks 27–32: Phase 4 Validation & Decommissioning
- Ongoing: Post-migration optimization & governance

Adjust scale & duration based on repo count, complexity, regulatory terms and team bandwidth.

---

16. Runbook (cutover checklist & emergency steps)

Pre-cutover checklist (per repo)

- Owner confirms migration request in backlog.
- Repo mirrored and commit/branch/tag parity verified.
- Converted workflow PR created and approved by owner.
- Secrets and environment secrets provisioned or OIDC confirmed.
- Branch protection and CODEOWNERS configured.
- Monitoring/alerts connected.

- Rollback plan reviewed and contacts listed.

Cutover steps (per repo)

1. Merge converted workflow PR and enable workflow (but do not remove ADO trigger).
2. Enable dual-run: set GitHub Actions to trigger on PR/commit while ADO continues.
3. Monitor for defined window (e.g., 48–72 hrs). Run validation suite.
4. If stable, disable ADO triggers and update downstream integrations to point to GitHub (webhooks, service hooks).
5. Move to decommission list for ADO.

Emergency rollback

- Disable GitHub workflow triggers and re-enable ADO pipeline trigger.
- If deployment incident: execute application rollback strategy (feature flags/blue-green/previous release).
- Rotate suspicious secrets and notify security.
- Open incident ticket and notify governance board.

Contacts & escalation

- Migration lead
- Platform SRE on-call
- Security lead
- App owner contacts (primary + backup)
- GitHub Enterprise support contact

---

17. Governance & long-term maintainability checklist

- IaC-first approach for org settings and policies (Terraform).
- Centralized reusable workflows, composite actions, and an internal action registry with vetting process.
- Periodic policy enforcement: scheduled checks for unpinned actions, secret scanning alerts, and branch protection drift.
- Quarterly security reviews: OIDC trust review, secret auditing, and action allowlist updates.
- Developer onboarding program continuously updated with samples and training.
- SLA for Platform team support during migrations and post-cutover.

---

18. Appendix: Quick mapping & conversion cheat-sheet

- Repos: git clone --mirror; git remote add github; git push --mirror github; handle git-lfs separately
- Pipelines: Azure pool -> runs-on, tasks -> actions/setup-*, publish -> actions/upload-artifact
- Variable groups -> GitHub org/repo/env secrets or external vault
- Service connections -> OIDC for cloud; fine-grained PATs only when necessary
- Artifacts -> GitHub Packages or GHCR
- Boards -> GitHub Issues/Projects; use migration tools and manual mapping for custom fields
- Classic UI pipelines -> convert to YAML workflows and commit to repo

- Hosted agents -> GitHub-hosted runners; special hardware -> self-hosted runners

---

19. Suggested checklist of deliverables to produce before migration

- Master inventory and prioritized migration backlog
- Org IaC repository (Terraform) with baseline org settings
- Template repository with reusable workflows and composite actions
- Conversion scripts and PR generator
- Validation automation suite and migration dashboard
- Migration runbook and rollback procedures
- Training materials and office-hours schedule
- Final decommission schedule and archive plan for ADO

If you want, next steps I can deliver:

- A tailored migration plan using your actual inventory (repo count, pipeline complexity, compliance requirements).
- A conversion toolkit skeleton (gh + Terraform + bash/python) that:
  - Creates repo, mirrors git + LFS, opens converted-workflow PRs, and runs validation checks.
- Example converted workflows for primary stacks (dotnet, Node.js, Java/Maven, Python) and a reusable-workflow library you can drop into .github.

If you choose a tailored plan, provide:

- Repo count and size distribution
- Number of pipelines (YAML vs classic) and % using custom tasks
- Number of packages/feeds and types
- Regulatory/compliance constraints and retention requirements
- Preferred timeline and blackout windows for production

This blueprint gives you a phased, auditable and secure path to move from Azure DevOps to GitHub Actions with minimal disruption and strong governance.